

EXPERIMENT-1 (question 1)

Name: S Rikhil

Roll_no: 2520030480

Section: 8

Experiment No. 1 (question 1)

Title: Installation of Linux VM, GCC Configuration, Git Repository Setup, Shell Architecture Analysis, and Process Creation using `fork()`, `exec()`, and System Call Tracing.

Aim: To set up a Linux development environment (VM, GCC, Git, Makefile, Project Structure) and write a C program to demonstrate process abstraction, execution of the `exec()` family, parent-child relationships, process tree inspection, and system call tracing using `strace`.

Objective:

- To set up a functional Linux environment, build system (`make`), and revision control (`git`).
- To understand shell architecture and process abstraction in operating systems.
- To learn process creation using the `fork()` system call.
- To execute external binaries using the `exec()` family of system calls (`exec1p`, `execvp`).
- To observe process relationships (parent-child, process trees using `ps`).
- To trace and analyze system calls made by a process using tools like `strace`.

• Theory:

• Linux Development Environment & Shell Architecture:

- **GCC & Makefile:** GCC (GNU Compiler Collection) compiles C programs into executable binaries. A Makefile automates the compilation process using rules and target dependencies.
- **Git:** A distributed version control system used to track changes in code files and manage project structure.
- **Shell Architecture:** The Linux shell acts as a command-line interface (CLI) that reads user input, parses commands, creates child processes using

`fork()`, replaces process images using `exec()`, and waits for completion using `wait()`.

- **Process Abstraction & Management System Calls:**

1. `fork()`:

- Creates a new child process.
- The child process is an exact copy of the parent.
- Returns:
 - 0 to the child process.
 - Child PID to the parent process.
 - -1 if creation fails.

2. `exec()`:

The `exec()` family replaces the current process image with a new program.

Common functions are:

- `execl()`
- `execv()`
- `execvp()`
- `execvp()`

In this experiment, the child process executes the user-entered Linux command.

3. `wait()`:

The parent process waits until the child process finishes execution.

This prevents zombie processes.

Algorithm:

1. Start the program.
2. Accept a Linux command from the user.
3. Create a child process using `fork()`.
4. If child process:
 - Execute the command using `execlp()` or `execvp()`.
5. If parent process:
 - Wait for the child process using `wait()`.

6. Display Parent PID and Child PID.
7. Stop.

System Calls Used:

Command / Call	Purpose
<code>fork()</code>	Creates a new child process.
<code>execvp()</code> / <code>execlp()</code>	Replaces current process with a target command.
<code>wait()</code>	Suspends parent execution until child terminates.
<code>getpid()</code> / <code>getppid()</code>	Returns current PID / parent PID.
<code>strace</code>	Intercepts and logs system calls executed by a program.
<code>pstree</code>	Displays process hierarchy as a tree diagram.
<code>make</code>	Automates compilation based on <code>Makefile</code> instructions.

Sample Output:

Passing Students: {'Alice': 85, 'Charlie': 92, 'Evan': 78}

Average Score: 85.00

- **Algorithm:**
 - Initialize project directory structure and initialize a Git repository using `git init`.
 - Create a `Makefile` to compile source code with `gcc`.
 - Read the user input (command name).
 - Call `fork()` to spawn a child process.
 - **Child Process (PID == 0):**
 - Display child PID and parent PID.
 - Execute command using `execlp()` / `execvp()`.
 - **Parent Process (PID > 0):**
 - Display parent PID and created child PID.
 - Call `wait(NULL)` to block until the child finishes.
 - Run the executable with `strace` to observe low-level system call sequences (`clone/fork`, `execve`, `wait4`).

Result:

The program was successfully executed. The Linux command entered by the user was executed through a child process. The parent process waited for the child process to complete execution, and the Process IDs of both parent and child were displayed successfully.

Code:

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Parent process started (PID: %d)\n", getpid());

    pid = fork(); // create child process

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("Child process (PID: %d, Parent PID: %d)\n", getpid(), getppid());

        // Replace child with "ls" program using exec
        execlp("ls", "ls", "-l", NULL);
    }
}
```

```

        // If exec fails
        perror("Exec failed");
    }
    else {
        // Parent process waits for child
        wait(NULL);

        printf("Parent process (PID: %d) finished waiting for child.\n", getpid());
    }

    return 0;
}

```

OUTPUT:

```

nissy@tuntun:~$ ./17120
Parent process started (PID: 502)
Child process (PID: 503, Parent PID: 502)
total 100
-rw-r--r-- 1 nissy nissy    16 Jul 30 05:01 1.txt
-rw-r--r-- 1 nissy nissy     0 Jul 30 15:23 command.c
-rw-r--r-- 1 nissy nissy     2 Jul 30 15:21 command.c.save
-rw-r--r-- 1 nissy nissy  477 Jul 30 15:27 command.c.save.1
-rw----- 1 nissy nissy     1 Jul 30 05:36 dat.c.save
-rw----- 1 nissy nissy  176 Jul 30 05:53 dat.c.save.1
-rw-r--r-- 1 nissy nissy     2 Jul 30 05:42 exp.c
-rw-r--r-- 1 nissy nissy     1 Jul 30 06:01 fork_exec.c
-rwxr-xr-x 1 nissy nissy 15952 Jul 30 05:25 hello
-rw-r--r-- 1 nissy nissy    76 Jul 30 05:04 hello.c
-rw-r--r-- 1 nissy nissy  2424 Jul 30 15:31 jes.c
-rw-r--r-- 1 nissy nissy  1454 Jul 30 15:31 jes.c.save
-rwxr-xr-x 1 nissy nissy 16344 Jul 30 15:32 nan
-rw-r--r-- 1 nissy nissy   794 Jul 30 15:31 nan.c
-rwxr-xr-x 1 nissy nissy 16208 Aug  1 10:03 nis
-rw-r--r-- 1 nissy nissy   765 Aug  1 10:03 nis.c
drwxr-xr-x 2 nissy nissy  4096 Jul 30 04:57 nissy
Parent process (PID: 502) finished waiting for child.
nissy@tuntun:~$ _

```